СОДЕРЖАНИЕ

Введение 6
1 Обзор литературы 7
1.1 Анализ современных архитектурных подходов 7
1.2 Паттерны управления микросервисной инфраструктурой 8
1.3 Анализ технологий межсервисного взаимодействия 10
1.4 Сравнительный анализ систем управления базами данных и кэширования 12
1.5 Обзор решений для обеспечения безопасности и SSO 13
1.6 Анализ систем мониторинга и наблюдаемости 15
1.7 Выбор средств контейнеризации и развертывания 17
2 Системное проектирование 20
3 Функциональное проектирование 21
3.1 Подраздел 21
3.2 Подраздел 21
3.3 Подраздел 21
3.4 Подраздел 21
3.5 Подраздел 21
3.6 Подраздел 21
3.7 Подраздел 21
3.8 Подраздел 21
4 Разработка программных модулей 22
5 Программа и методика испытаний 23
5.1 Подраздел 23
5.2 Подраздел 23
5.3 Подраздел 23
6 Руководство пользователя 24
6.1 Подраздел 24
6.2 Подраздел 24
6.3 Подраздел 24
7 Технико-экономическое обоснование разработки и реализации на рынке xxxxxxx 25
7.1 Подраздел 25
7.2 Подраздел 25
7.3 Подраздел 25
7.4 Подраздел 25
7.5 Подраздел 25
Заключение 26
Список использованных источников 27
Приложение А 29
Приложение Б 30
Приложение В 31
Приложение Г 32
Приложение Д 33

ВВЕДЕНИЕ

В современном мире информационные технологии пронизывают практически все сферы человеческой деятельности, определяя скорость развития бизнеса, качество услуг и конкурентоспособность компаний. Одним из наиболее динамично развивающихся направлений является микросервисная архитектура – подход к построению приложений в виде набора небольших, независимых сервисов, каждый из которых отвечает за конкретную бизнес-функцию и может разрабатываться, развертываться и масштабироваться отдельно.

Микросервисная архитектура позволяет значительно повысить гибкость разработки, ускорить выход новых бизнес-ценностей на рынок, упростить поддержку и обновление систем, а также обеспечить высокую отказоустойчивость и масштабируемость под пиковые нагрузки. По данным различных исследований, в последние годы наблюдается устойчивый рост популярности этого подхода.

Особенно актуален микросервисный подход в высоконагруженных распределённых системах, таких как агрегаторы такси, маркетплейсы, финтех-сервисы и e-commerce платформы, где требуется обработка тысяч и миллионов запросов в реальном времени, высокая доступность и возможность независимого развития отдельных модулей.

Целью дипломного проектирования является разработка и развёртывание производственно-готовой инфраструктуры для микросервисного приложения агрегатора такси с базовой бизнес-логикой, обеспечивающей масштабируемую и удобную в эксплуатации основу для дальнейшей промышленной разработки.

Для достижения поставленной цели решаются следующие задачи:

– анализ современных практик и паттернов построения микросервисных систем;

– проектирование и настройка центральных инфраструктурных компонентов: шлюза, обнаружения сервисов, отказоустойчивости;

– организация системы единой авторизации и аутентификации;

– выбор и настройка хранилищ данных;

– реализация асинхронного взаимодействия между микросервисами;

– построение полноценной системы логирования и трассировки

– разработка и настройка базовых микросервисов, реализующих стандартные операции.

Практическая значимость разработанной инфраструктуры заключается в создании масштабируемой, отказоустойчивой и удобной в эксплуатации основы для высоконагруженных агрегаторов такси, что позволяет независимо развивать бизнес-функции (поиск водителя, расчёт стоимости, оплата и т.д.), минимизировать простои и ускорять вывод новых возможностей на рынок.

Дипломный проект выполнен мной лично, проверен на заимствования, процент оригинальности составляет **XX%** (отчёт о проверке на заимствования прилагается).

1 ОБЗОР ЛИТЕРАТУРЫ

1.1 Анализ современных архитектурных подходов

На сегодняшний день выбор архитектурного стиля является определяющим фактором при проектировании масштабируемых информационных систем. Традиционным подходом долгое время являлась монолитная архитектура, при которой все компоненты приложения (бизнес-логика, работа с данными, пользовательский интерфейс) объединены в единый программный блок и развертываются как одно целое [1].

Основными достоинствами монолитной архитектуры являются простота разработки на начальных этапах, легкость тестирования и развертывания. Однако по мере роста системы, особенно в таких высоконагруженных доменах, как агрегаторы такси, монолит становится трудноуправляемым. Внесение изменений в один модуль требует сборки всего приложения, а невозможность масштабировать отдельные компоненты (например, только модуль расчета стоимости поездки) ведет к нерациональному использованию ресурсов.

В качестве альтернативы выступает микросервисная архитектура – подход, при котором приложение разбивается на набор слабосвязанных сервисов, взаимодействующих по сети. Каждый сервис отвечает за конкретную бизнес-способность и владеет собственной базой данных.

Разница между монолитной и микросервисной архитектурой представлена на рисунке 1.1.

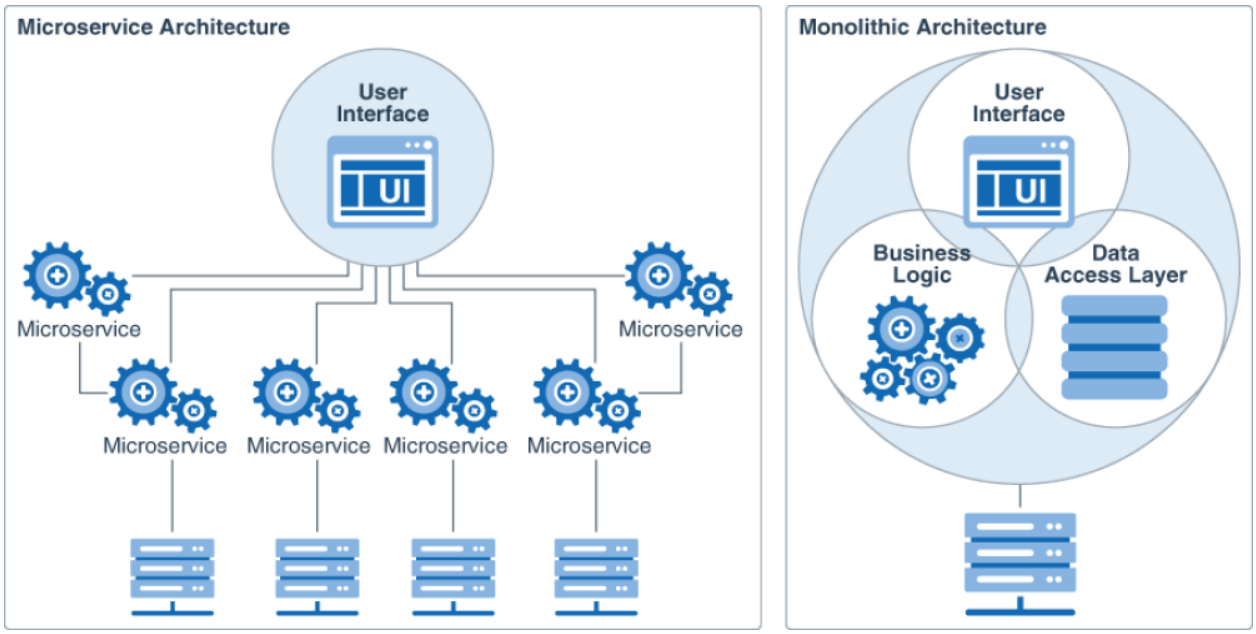

Рисунок 1.1 – Различия между монолитной и микросервисной архитектуры [1]

Согласно исследованию Криса Ричардсона [2], ключевыми

преимуществами микросервисного подхода являются:

– независимость развертывания;

– возможность использования различных технологических стеков для разных задач;

– изоляция сбоев (отказ одного сервиса не приводит к полной остановке системы);

– легкое горизонтальное масштабирование наиболее нагруженных узлов.

Однако переход к микросервисам привносит сложность в управлении инфраструктурой. Возникают задачи обнаружения сервисов, балансировки нагрузки, обеспечения безопасности распределенных запросов и мониторинга состояния множества узлов.

1.2 Паттерны управления микросервисной инфраструктурой

Разделение приложения на десятки независимых сервисов порождает ряд проблем: как клиенту (мобильному приложению такси) узнать адреса всех сервисов, как обеспечить безопасность на входе и как предотвратить «эффект домино» при отказе одного из узлов. Для решения этих задач в современной практике проектирования выделяют три ключевых паттерна.

1.2.1 Паттерн API Gateway [3] представляет из себя наличие отдельного компонента, который выступает единой точкой входа для всех внешних запросов. Вместо того чтобы мобильное приложение или веб-интерфейс взаимодействовали с каждым микросервисом напрямую, они отправляют запрос на шлюз.

Помимо базовой маршрутизации, современный API Gateway выполняет ряд критически важных функций:

1 Централизованная безопасность. Шлюз берет на себя задачу терминации SSL/TLS-соединений, избавляя внутренние микросервисы от необходимости обрабатывать сертификаты. Здесь же происходит проверка JWT-токенов или API-ключей. Это гарантирует, что неавторизованный трафик даже не попадет во внутренний контур сети.

2 Ограничение частоты запросов. Для агрегатора такси крайне важна защита от перегрузок или злонамеренных действий. Шлюз позволяет устанавливать лимиты на количество запросов в секунду от конкретного пользователя или IP-адреса.

3 Кэширование ответов. Для запросов, данные в которых меняются редко, шлюз может возвращать кэшированный ответ, значительно снижая нагрузку на нижележащие сервисы.

4 Агрегация запросов. Чтобы снизить количество сетевых вызовов от мобильного приложения, шлюз может принять один запрос, одновременно вызвать несколько микросервисов и вернуть клиенту объединенный результат.

Использование шлюза также упрощает мониторинг системы, так как

именно здесь можно собирать общую статистику по времени ответа и количеству ошибок во всей распределенной системе.

На рисунке 1.2 представлена схема взаимодействия сервисов через API Gateway.

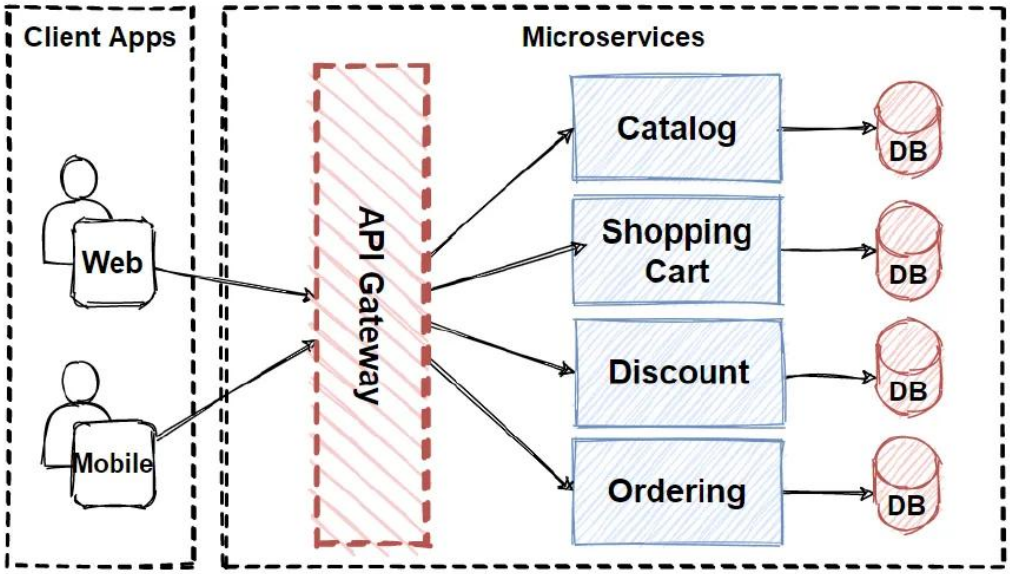

Рисунок 1.2 – Пример взаимодействия сервисов через API Gateway [3]

1.2.2 Паттерн Service Discovery [4] позволяет сервисам находить друг друга без жестко прописанных адресов в конфигурации.

Система обнаружения состоит из двух частей: service registry (реестр), который представляет из себя базу данных, в которой хранится информация о доступных экземплярах сервисов и их адресах, и процесс регистрации, в котором микросервис сообщает реестру свой адрес, а при остановке – удаляет (рисунок 1.3).

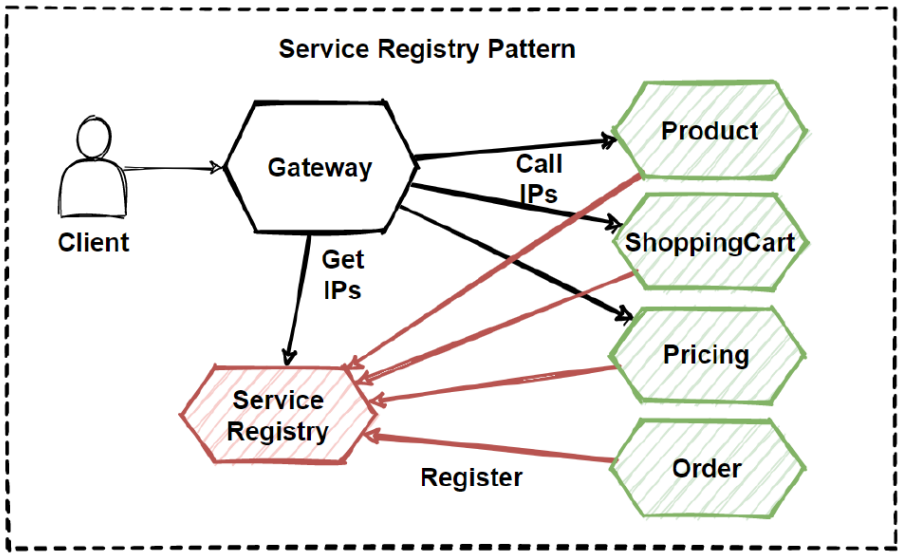

Рисунок 1.4 – Принцип работы паттерна Service Discovery

Использование этого паттерна критически важно для приложений, где нагрузка может резко возрастать, что требует мгновенного запуска новых копий сервиса и их автоматического включения в работу системы.

1.2.3 Паттерн Circuit Breaker [5] позволяет легче справляться с отказами одного из сервиса в распределенной системе, так как отказы неизбежны. Если сервис А начинает отвечать с задержкой, запросы от сервиса В могут начать скапливаться, занимая память и потоки процессора, что в итоге приведет к падению всей системы. Данный паттерн позволяет избегать такие каскадные сбои.

Паттерн работает по принципу электрического автомата и имеет три состояния (рисунок 1.4):

– closed (закрыт), когда запросы проходят в обычном режиме;

– open (открыт), если количество ошибок превышает порог, шлюз сразу возвращает ошибку клиенту, не пытаясь вызвать неисправный сервис;

– half-open (полуоткрыт), когда через некоторое время шлюз пропускает пробный запрос, чтобы проверить, восстановился ли сервис.

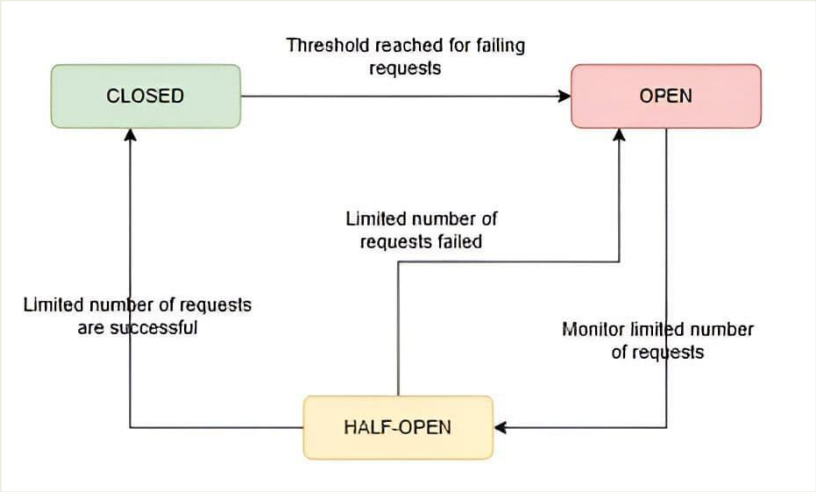

Рисунок 1.4 – Принцип работы паттерна Circuit Breaker [5]

Применение этого паттерна гарантирует, что даже если в приложении не работает хоть один сервис, пользователи всё равно смогут продолжать работу с системой в целом, если неработающий сервис не затронул функционал, который необходим пользователю.

1.3 Анализ технологий межсервисного взаимодействия

В микросервисной архитектуре основное разделение взаимодействия между сервисами происходит на два типа: синхронное и асинхронное взаимодействие.

1.3.1 Синхронное взаимодействие подразумевает, что сервис-

отправитель ожидает немедленного ответа от сервиса-получателя. Наиболее распространенным протоколом здесь является REST (HTTP/JSON).

Преимущество синхронного подхода заключается в простоте реализации и отладки, и получение мгновенного результата выполнения операции.

Однако это создает большую связность между сервисами микросервисной системы, что негативно сказывается на общей производительности системы.

1.3.2 Асинхронное взаимодействие и событийно-ориентированная архитектура строится на передачи сообщений через промежуточное звено – брокер сообщений. В этом случае сервис-отправитель просто отправляет сообщение (событие) в очередь и продолжает свою работу, не дожидаясь обработки.

Сравнение двух способов взаимодействия представлено на рисунке 1.5.

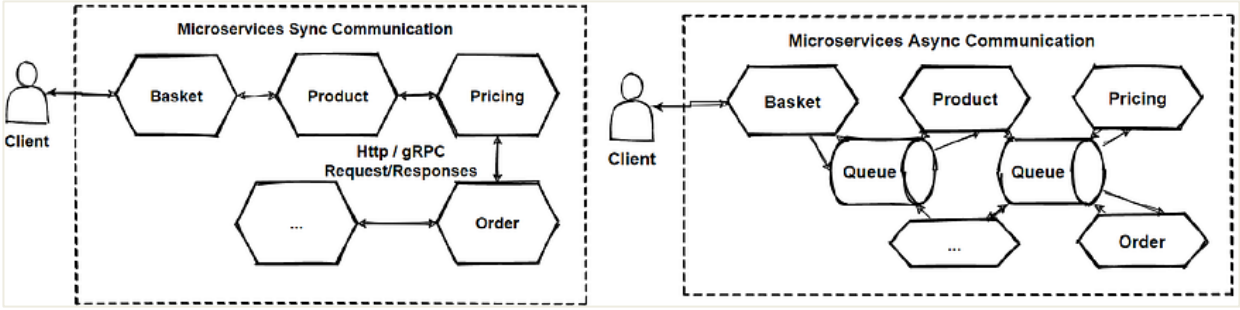

Рисунок 1.5 – Сравнение синхронного и асинхронного способа взаимодействия в микросервисной архитектуре [6]

Ключевые преимущества использования брокеров сообщений в крупных микросервисных инфраструктурах:

1 Сглаживание пиковых нагрузок. В праздничные дни количество заказов резко возрастает. Брокер выступает буфером, позволяя сервисам обрабатывать очередь сообщений с той скоростью, на которую они рассчитаны, не падая от перегрузки.

2 Гарантия доставки. Если сервис-обработчик временно отключен, сообщения накопятся в брокере и будут обработаны сразу после его включения.

3 Слабая связность. Сервис A ничего не знает о существовании сервиса B. Он публикует событие, а любые другие сервисы могут подписаться на это событие самостоятельно [6].

1.3.3 Технология Apache Kafka [7] используется для реализации асинхронного обмена данными в проекте. В отличие от классических очередей (например, RabbitMQ), Kafka является распределенной платформой потоковой передачи событий.

Основные отличия Kafka, важные для инфраструктуры агрегатора такси:

1 Высокая пропускная способность. Способность обрабатывать миллионы событий в секунду, что критично для отслеживания геолокации тысяч автомобилей в реальном времени.

2 Хранение данных. Kafka записывает сообщения на диск и хранит их в течение заданного времени. Это позволяет проводить «повтор событий» для восстановления состояния системы и проведения аналитики.

3 Масштабируемость. Система легко расширяется добавлением новых брокеров, что позволяет инфраструктуре расти вместе с базой пользователей агрегатора.

1.4 Сравнительный анализ систем управления базами данных и кэширования

В микросервисной архитектуре применяется паттерн «база данных на каждый сервис», что гарантирует слабую связность компонентов и независимость их масштабирования. Для агрегатора такси критически важными требованиями к хранилищам данных являются обеспечение строгой консистентности финансовых транзакций (оплата поездок), надежное хранение профилей пользователей, а также возможность быстрой обработки геопространственных данных.

1.4.1 В качестве основной СУБД выступает PostgreSQL [8], которая является объектно-реляционной системой. Ключевыми преимуществами PostgreSQL для инфраструктуры агрегатора такси являются:

1 Строгое соответствие принципам ACID (атомарность, согласованность, изолированность, долговечность), что критически важно для микросервисов, отвечающих за биллинг и расчеты.

2 Расширенная поддержка JSON/JSONB. Это позволяет гибко хранить слабоструктурированные данные (например, логи изменений статусов заказов) в рамках реляционной модели, частично заменяя необходимость использования NoSQL-решений (таких как MongoDB).

3 Наличие расширения PostGIS. Данное расширение делает PostgreSQL мощным инструментом для обработки геопространственных данных, (что является ядром бизнес-логики любого агрегатора такси.

Помимо надежного хранения, в высоконагруженных системах возникает необходимость минимизации времени отклика и снижения нагрузки на основную БД при частых запросах одних и тех же данных (например, тарифы, текущие сессии пользователей, статусы активности водителей). Для этого применяются системы кэширования в оперативной памяти (In-Memory Data Grid).

Наиболее распространенными решениями в этой области являются Memcached и Redis.

Memcached представляет собой простую и высокопроизводительную распределенную систему кэширования объектов в памяти на основе хэш-

таблиц (ключ-значение). Она отлично справляется с базовым кэшированием строковых данных, но не поддерживает сложные структуры данных.

Redis, в отличие от Memcached, является сервером структур данных. Его основные преимущества для проектируемой системы:

1 Поддержка сложных типов данных: списков, множеств, хэшей, что позволяет реализовывать сложные алгоритмы кэширования на стороне БД.

2 Встроенная поддержка геопространственных индексов, что позволяет хранить и мгновенно обновлять текущие координаты водителей в оперативной памяти, не перегружая дисковую подсистему PostgreSQL.

3 Возможность сохранения данных на диск, что обеспечивает быстрое восстановление состояния кэша после перезагрузки узла.

На основе проведенного сравнительного анализа для разрабатываемой микросервисной инфраструктуры в качестве основной реляционной СУБД для хранения бизнес-сущностей выбрана PostgreSQL, а в качестве in-memory хранилища для кэширования и управления быстроменяющимися состояниями – Redis. Данная связка является индустриальным стандартом и обеспечивает необходимый баланс между надежностью, масштабируемостью и производительностью.

1.5 Обзор решений для обеспечения безопасности и SSO

В микросервисной архитектуре агрегатора такси проблема аутентификации и авторизации требует особого подхода. Поскольку система состоит из множества независимых сервисов, передача учетных данных пользователя (логина и пароля) каждому из них небезопасна и неэффективна. Для решения этой задачи применяется концепция единого входа (Single Sign-On, SSO) и централизованного управления доступом. Основными стандартами для реализации безопасного делегированного доступа на сегодняшний день являются протоколы OAuth 2.0 и OpenID Connect (OIDC) [9].

1.5.1 Облачные провайдеры аутентификации (IDaaS) [10] – сервисы, предоставляющие функционал управления доступом по модели SaaS. Наиболее популярными представителями являются Auth0, Firebase Authentication и AWS Cognito. Данные платформы берут на себя всю сложность реализации криптографии, хранения паролей и масштабирования.

Сервисы данного типа обладают следующими достоинствами:

– максимально быстрое внедрение и интеграция;

– готовый пользовательский интерфейс для регистрации, входа и восстановления пароля;

– высокая отказоустойчивость, обеспечиваемая поставщиком услуг;

– встроенные механизмы защиты от атак и многофакторная аутентификация.

Однако облачные сервисы имеют существенные недостатки:

– хранение персональных данных пользователей на сторонних, зачастую зарубежных серверах, что может нарушать законодательные требования о

локализации данных;

– привязка к экосистеме конкретного провайдера;

– высокая стоимость эксплуатации при росте активной пользовательской базы (оплата за количество уникальных пользователей).

1.5.2 Фреймворки для создания собственных серверов авторизации [11]. Если проект требует уникальной логики аутентификации, разработчики могут реализовать сервер с нуля, используя специализированные библиотеки, такие как IdentityServer (для экосистемы .NET) или Spring Security Authorization Server (для Java).

Достоинства подхода:

– полный контроль над логикой работы, алгоритмами шифрования и процессом выдачи токенов;

– возможность интеграции с любыми устаревшими базами данных.

Недостатки данного подхода:

– колоссальные временные затраты на разработку, тестирование и поддержку;

– высокие риски допущения критических уязвимостей в безопасности из-за человеческого фактора;

– необходимость самостоятельной разработки панелей администрирования пользователей.

1.5.3 Готовые решения для локального развертывания. Этот подход объединяет преимущества облачных платформ (готовый функционал) и собственной разработки (полный контроль над данными). Самым популярным решением в этой категории является Keycloak [12] – продукт с открытым исходным кодом, разрабатываемый при поддержке Red Hat.

Keycloak представляет собой полноценный сервер аутентификации и авторизации, который разворачивается в собственной инфраструктуре проекта.

Его основные достоинства:

– бесплатное использование без ограничений на количество пользователей;

– поддержка всех современных протоколов (OAuth 2.0, OpenID Connect, SAML);

– готовая реализация SSO, позволяющая пользователям авторизоваться один раз для доступа ко всем подсистемам агрегатора;

– гибкая настройка ролевой модели доступа (RBAC) и панель администратора «из коробки»;

– полный контроль над данными, так как база данных с учетными записями находится во внутреннем изолированном контуре инфраструктуры и недоступна извне.

В таблице 1.3 представлен сравнительный анализ рассмотренных подходов, который позволяет выбрать оптимальный подход к вопросам аутентификации и авторизации.

Таблица 1.1 – Сравнение решений для обеспечения безопасности и SSO

Решение	Тип	Достоинства	Недостатки
Auth0 или Firebase	Облачный сервис (SaaS)	Быстрый старт, высокая надежность, не требует поддержки серверов	Риски хранения персональных данных у третьих лиц, высокая цена при масштабировании
IdentityServer или Spring Authorization Server	Библиотека	Абсолютный контроль над логикой, глубокая кастомизация	Высокая стоимость разработки, риск уязвимостей, требует узких специалистов
Keycloak	Готовый сервер	Бесплатный функционал SSO «из коробки», полный контроль над данными	Требует ресурсов для локального хостинга и настройки инфраструктуры

Таким образом, в контексте разработки микросервисной инфраструктуры для агрегатора такси, где критически важны безопасность, независимость от сторонних вендоров и защита персональных данных, подход с использованием сторонних облачных API нецелесообразен. Разработка собственного сервера требует неоправданно высоких трудозатрат. Решение Keycloak является наиболее сбалансированным выбором, предоставляя стандартизированный функционал уровня Enterprise при полном контроле над развертыванием и хранением данных.

1.6 Анализ систем мониторинга и наблюдаемости

Разделение монолитного приложения на десятки независимых микросервисов неизбежно усложняет процессы поиска неисправностей и контроля состояния системы. В случае сбоя администраторам сложно определить, в каком именно узле или базе данных произошла ошибка. Для решения этой проблемы внедряется концепция «наблюдаемости» (Observability), базирующаяся на трех основных компонентах: метриках, логировании и распределенной трассировке.

1.6.1 Системы сбора метрик представляют из себя инструменты, которые собирают числовые показатели работы системы (потребление CPU, оперативной памяти, количество HTTP-запросов в секунду).

Традиционным решением для мониторинга серверов является система Zabbix. Она использует push-модель, при которой агенты, установленные на

серверах, регулярно отправляют данные на центральный сервер. Zabbix отлично подходит для статической инфраструктуры, однако в динамических микросервисных средах (где Docker-контейнеры могут создаваться и удаляться ежеминутно) управление агентами становится затруднительным.

В качестве индустриального стандарта для контейнеризированных приложений выступает Prometheus [13]. Его ключевым отличием является pull-модель: сервер мониторинга самостоятельно опрашивает метрики с микросервисов по HTTP-протоколу. Prometheus обладает встроенной базой данных временных рядов и мощным языком запросов PromQL, что делает его оптимальным выбором для агрегатора такси.

1.6.2 Системы централизированного логирования представляют из себя системы, которые содержат детальную текстовую информацию о событиях внутри приложения. Поскольку микросервисы распределены по разным контейнерам, необходимо собирать их логи в единое хранилище.

Исторически самым популярным решением является стек ELK (Elasticsearch, Logstash, Kibana) [14]. Elasticsearch обеспечивает мощный полнотекстовый поиск по всем логам, а Kibana предоставляет удобный веб-интерфейс для аналитики (рисунок 1.6).

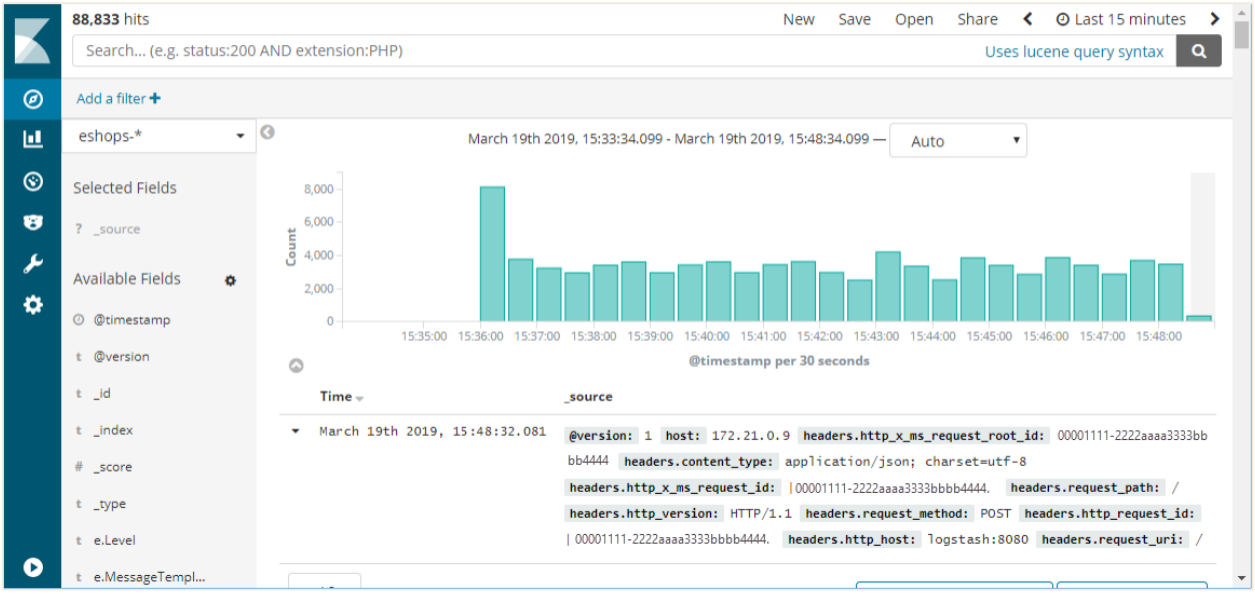

Рисунок 1.6 – Веб-интерфейс Kibana

Несмотря на высокую функциональность, главным недостатком стека ELK является колоссальное потребление вычислительных ресурсов. Индексация полного текста каждого сообщения требует значительных объемов оперативной памяти, что избыточно для базовой инфраструктуры.

Современной и легковесной альтернативой является стек LOKI [15], разработанный компанией Grafana Labs. В отличие от Elasticsearch, Loki не индексирует содержимое логов, а индексирует только метаданные, аналогично подходу Prometheus. Текст логов сжимается и хранится в

объектном хранилище. Это позволяет кардинально снизить требования к ресурсам серверов (CPU и RAM), сохраняя при этом высокую скорость поиска по нужным ярлыкам. Веб-интерфейс Grafana представлен на рисунке 1.7.

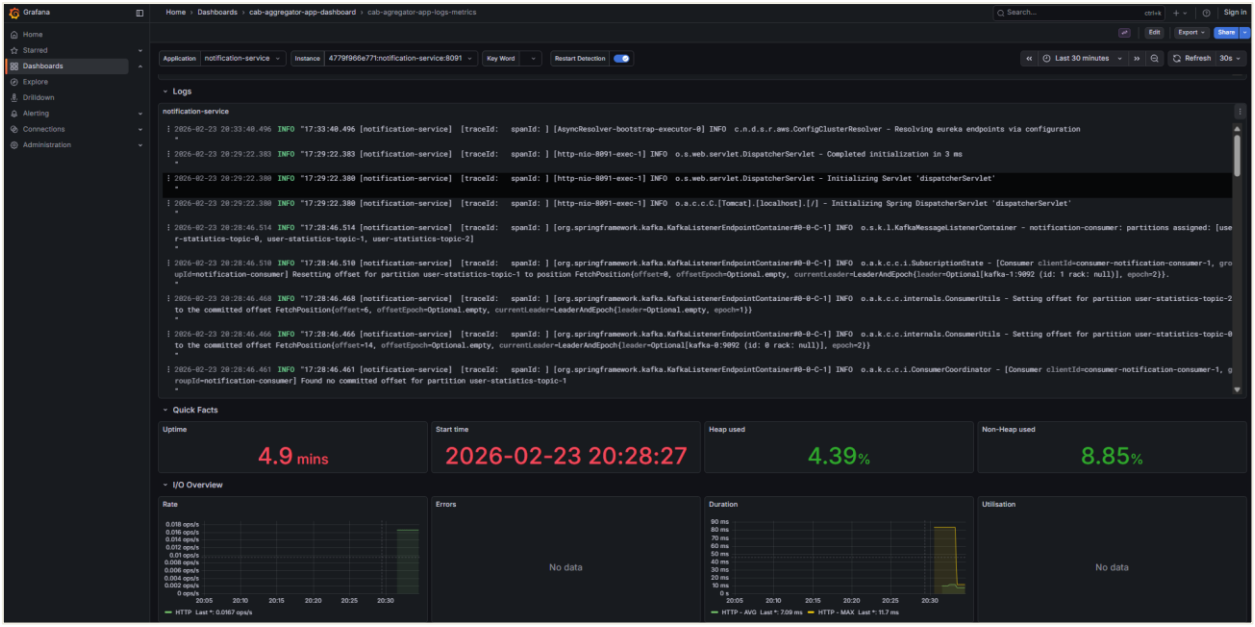

Рисунок 1.7 – Веб-интерфейс Grafana

1.6.3 Система распределенной трассировки позволяет отследить путь одного пользовательского запроса и выявить узкие места производительности. Ранее для этих целей использовались системы Jaeger или Zipkin со своими проприетарными агентами. На сегодняшний день стандартом де-факто стал проект OpenTelemetry [16] – единый набор API и SDK для инструментирования кода. OpenTelementry позволяет собирать трейсы и отправлять их в любой backend.

В качестве хранилища трейсов для проекта выбрана система Tempo. Она глубоко интегрирована с Loki и Prometheus, отличается высокой скоростью записи и не требует сложной настройки баз данных для своей работы.

Исходя из всей информации выше оптимальный решением в данной инфраструктуре является использование LOKI стека, как более современный и легковесный, при этом не уступающий по функционалу ELK стеку.

1.7 Выбор средств контейнеризации и развертывания

Разработка масштабируемой микросервисной архитектуры агрегатора такси, включающей базы данных (PostgreSQL, Redis), брокер сообщений (Kafka), сервер авторизации (Keycloak) и системы мониторинга, требует унифицированного подхода к развертыванию. Главная проблема при переносе такого количества компонентов между средами заключается в конфликтах зависимостей и различиях в конфигурации операционных систем.

Для решения этой проблемы применяются технологии изоляции сред

выполнения. На сегодняшний день выделяют несколько основных подходов к развертыванию информационных систем.

1.7.1 Традиционная виртуализация через виртуальные машины на базе гипервизоров, таких как VMware, VirtualBox, KVM является первым способом изоляции сред. Каждая ВМ эмулирует аппаратное обеспечение и содержит полноценную гостевую операционную систему. Основное достоинство данного подхода – это высочайший уровень изоляции и безопасности.

Недостатки:

– колоссальные накладные расходы вычислительных ресурсов (CPU, RAM, дисковое пространство) на поддержание работы множества ОС;

– долгое время запуска, что критично для микросервисов, которые должны быстро масштабироваться при росте нагрузки.

1.7.2 В отличие от виртуальных машин, контейнеризация на уровне ОС не эмулирует аппаратное обеспечение, а используют общее ядро операционной системы хоста, изолируя процессы на уровне пространств имен (namespaces) и контрольных групп (cgroups) Linux. Это позволяет запускать сервисы за миллисекунды и потреблять минимум ресурсов.

Среди платформ контейнеризации рассматриваются две основные:

1 Podman [17] – современная утилита от Red Hat. Ее главное преимущество заключается в архитектуре без демона и возможности запуска контейнеров без root-прав, что повышает безопасность. Однако инструмент имеет более сложную настройку сетевого взаимодействия и меньшее количество готовых интеграций.

2 Docker [18]– индустриальный стандарт де-факто для контейнеризации. Docker предоставляет удобный CLI-интерфейс, огромную экосистему готовых образов (Docker Hub) и отличается высокой стабильностью. В контексте данной инфраструктуры, где требуется объединить множество готовых инфраструктурных решений, Docker является наиболее надежным и задокументированным инструментом.

1.7.3 Средства оркестрации контейнеров позволяют управлять многоконтейнерными приложениями на разных узлах, при это описывая этот процесс декларативно.

Для решения этой задачи рассматриваются системы оркестрации:

1 Kubernetes [19] – мощнейшая платформа для управления кластерами контейнеров. Она обеспечивает автоматическое масштабирование, самовосстановление и балансировку нагрузки. Недостатком K8s является чрезвычайно высокий порог входа, сложность архитектуры и высокие требования к ресурсам. Использование Kubernetes для базового развертывания инфраструктуры на текущем этапе является избыточным решением.

2 Docker Compose [20] – утилита для локального развертывания и связывания контейнеров с использованием формата YAML. С помощью одного конфигурационного файла docker-compose.yaml и одной команды

можно развернуть всю спроектированную инфраструктуру агрегатора со всеми зависимостями.

Важнейшим архитектурным преимуществом выбора формата Docker Compose является его прямая совместимость с нативным оркестратором Docker Swarm [21]. Если в будущем проекту потребуется распределение нагрузки между несколькими физическими серверами (создание кластера), конфигурационные файлы docker-compose.yaml имеют полную обратную совместимость с Docker Swarm.

Таким образом, для упаковки всех компонентов приложения выбран Docker, а в качестве инструмента декларативного описания и запуска производственно-готовой инфраструктуры – Docker Compose с перспективой прозрачного масштабирования через Docker Swarm.

2 СИСТЕМНОЕ ПРОЕКТИРОВАНИЕ

Текст раздела

3 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ

3.1 Подраздел

Текст раздела

3.2 Подраздел

Текст раздела

3.3 Подраздел

Текст раздела

3.4 Подраздел

Текст раздела

3.5 Подраздел

Текст раздела

3.6 Подраздел

Текст раздела

3.7 Подраздел

Текст раздела

3.8 Подраздел

Текст раздела

4 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ

Текст раздела

5 ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ

Текст раздела

5.1 Подраздел

Текст раздела

5.2 Подраздел

Текст раздела

5.3 Подраздел

Текст раздела

6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Текст раздела

6.1 Подраздел

Текст раздела

6.2 Подраздел

Текст раздела

6.3 Подраздел

Текст раздела

7 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ И РЕАЛИЗАЦИИ НА РЫНКЕ XXXXXXX

7.1 Подраздел

Текст раздела

7.2 Подраздел

Текст раздела

7.3 Подраздел

Текст раздела

7.4 Подраздел

Текст раздела

7.5 Подраздел

Текст раздела

ЗАКЛЮЧЕНИЕ

Должно быть минимум на 81 странице!!

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Ричардсон, К. Микросервисы. Паттерны разработки и рефакторинга. / К. Ричардсон– СПб. : Питер, 2026. – 544 с.

[2] Сравнение монолитной и микросервисной архитектуры [Электронный ресурс]. – Режим доступа: https://proglib.io/p/monolitnaya-vs-mikroservisnaya-arhitektura-2019-09-16 – Дата доступа: 19.02.2026.

[3] API Gateway Pattern [Электронный ресурс]. – Режим доступа: https://medium.com/design-microservices-architecture-with-patterns/api-gateway-pattern-8ed0ddfce9df – Дата доступа: 19.02.2026.

[4] Service Discovery Pattern [Электронный ресурс]. – Режим доступа: https://medium.com/design-microservices-architecture-with-patterns/service-registry-pattern-75f9c4e50d09 – Дата доступа: 19.02.2026.

[5] Circuit Breaker Pattern [Электронный ресурс]. – Режим доступа: https://www.baeldung.com/cs/microservices-circuit-breaker-pattern – Дата доступа: 19.02.2026.

[6] Microservices Asynchronous Message-Based Communication [Электронный ресурс]. – Режим доступа: https://medium.com/design-microservices-architecture-with-patterns/microservices-asynchronous-message-based-communication-6643bee06123 – Дата доступа: 20.02.2026.

[7] Apache Kafka [Электронный ресурс]. – Режим доступа: https://kafka.apache.org – Дата доступа: 20.02.2026.

[8] PostgreSQL Documentation [Электронный ресурс]. – Режим доступа: https://www.postgresql.org/docs – Дата доступа: 23.02.2026.

[9] OAuth 2.0 и OpenID Connect [Электронный ресурс]. – Режим доступа: https://habr.com/ru/articles/916640/– Дата доступа: 23.02.2026.

[10] Identity as a Service [Электронный ресурс]. – Режим доступа: https://habr.com/ru/companies/mclouds/articles/329964 – Дата доступа: 23.02.2026.

[11] Spring Authorization Server Reference [Электронный ресурс]. – Режим доступа: https://docs.spring.io/spring-authorization-server/reference/index.html – Дата доступа: 23.02.2026.

[12] Keycloak [Электронный ресурс]. – Режим доступа: https://www.keycloak.org/guides – Дата доступа: 23.02.2026.

[13] Prometheus [Электронный ресурс]. – Режим доступа: https://prometheus.io/docs/introduction/overview – Дата доступа: 23.02.2026.

[14] ELK стек [Электронный ресурс]. – Режим доступа: https://habr.com/ru/articles/671344 – Дата доступа: 23.02.2026.

[15] LOKI стек [Электронный ресурс]. – Режим доступа: https://habr.com/ru/companies/badoo/articles/507718 – Дата доступа: 23.02.2026.

[16] OpenTelemetry [Электронный ресурс]. – Режим доступа: https://opentelemetry.io – Дата доступа: 23.02.2026.

[17] Podman Documentation [Электронный ресурс]. – Режим доступа: https://docs.podman.io/en/latest – Дата доступа: 23.02.2026.

[18] Docker Documentation [Электронный ресурс]. – Режим доступа:

https://docs.docker.com – Дата доступа: 23.02.2026.

[19] Kubernetes Documentation [Электронный ресурс]. – Режим доступа: https://kubernetes.io/docs/home – Дата доступа: 23.02.2026.

[20] Docker Compose Documentation [Электронный ресурс]. – Режим доступа: https://docs.docker.com/compose – Дата доступа: 23.02.2026.

[21] Docker Swarm Documentation [Электронный ресурс]. – Режим доступа: https://docs.docker.com/engine/swarm – Дата доступа: 23.02.2026.

Вычислительные машины, системы и сети: дипломное проектирование [Электронный ресурс]. – Электронные данные. – Режим доступа: https://www.bsuir.by/m/12_100229_1_136308.pdf – Дата доступа: 31.03.2026.

Экономика проектных решений: методические указания по экономическому обоснованию дипломных проектов [Электронный ресурс]. – Электронные данные. – Режим доступа: https://www.bsuir.by/m/12_100229_1_161144.pdf – Дата доступа: 31.03.2026.

ПРИЛОЖЕНИЕ А
(обязательное)

Листинг кода

```
Код начинается с этого места;
001  Можно с нумерацией строк кода;
002  Шрифт может быть от 12 до 8 pt
```

ПРИЛОЖЕНИЕ Б
(обязательное)

Название приложения

ПРИЛОЖЕНИЕ В
(обязательное)

Отчет о проверке на заимствования

ПРИЛОЖЕНИЕ Г
(обязательное)

Спецификация

ПРИЛОЖЕНИЕ Д
(обязательное)

Ведомость документов